Version 2.0

January 2026

OBJECT-ORIENTED PROGRAMMING

MODULE 5 - EXCEPTION HANDLING, JAVA COLLECTIONS

Author:

Ir. Galih Wasis Wicaksono, S.Kom., M.Cs.

Yossua Agung Budianto

Muhammad Budi Kusuma

INFORMATICS LABORATORY

University of Muhammadiyah Malang

TABLE OF CONTENTS

INTRODUCTION

OBJECTIVES

1. Understand the importance of error-handling mechanisms to build robust applications and prevent sudden system crashes.
2. Identify the hierarchy of issues in Java, ranging from high-level system failures (Error) to controllable program logic issues (Exception).
3. Understand the advantages of the Java Collections Framework in managing sets of objects dynamically and efficiently compared to the rigid nature of conventional arrays.

MODULE TARGETS

1. Students are able to implement try-catch-finally blocks to capture and handle various runtime anomalies.
2. Students are able to distinguish between and appropriately handle Checked Exceptions (external risks) and Unchecked Exceptions (logic errors).
3. Students are able to create and trigger Custom Exceptions to handle specific cases tailored to application business rules.
4. Students are able to use ArrayList to store data with capacity that automatically expands or shrinks.
5. Students are able to perform data traversing using the Enhanced For-Loop and Iterator for safe data modification.

PREPARATION

1. Muslim students may recite the following prayer before starting the lesson

   رَضِيْتُ بِاللهِ رَبًّا وَبِالإِسْلَامِ دِيْنًا وَبِمُحَمَّدٍ نَبِيًّا وَرَسُوْلًا. رَبِّ زِدْنِيْ عِلْمًا وَارْزُقْنِيْ فَهْمًا

2. Laptop or personal computer
3. Intellij IDE/Any Java Compiler (Eclipse VSCode)
4. “Full Poweeeerrrrrrrr”

KEYWORDS

Exception Handling, Throwable, Error, Exception, Checked Exception, Unchecked Exception, RuntimeException, Try, Catch, Finally, Throw, Throws, Stack Trace, Custom Exception, Java Collections Framework, ArrayList, Generics, Wrapper Class, Dynamic Sizing, Iterator, Enhanced For-Loop, ConcurrentModificationException.

THEORY

1. Exception Handling

1.1 Why Should I Understand This?

Suppose you are reading this module using a reader application and you try one of its features, for example "find word." Suddenly, your browser crashes or closes unexpectedly. After reopening the document in that app and searching for the word manually, it turns out the word you were looking for does not exist in the document, and you realize that the crash was caused by the failed feature. You think it would have been better if the application displayed a message like "word not found" instead of forcefully shutting down. This illustrates software without proper error handling.

Without exception handling, computer programs are very fragile because they assume everything will always go smoothly according to the ideal plan (**the happy path**). As soon as one small unexpected error occurs, the entire system stops working instantly because the computer doesn't know what to do other than "**give up**" and **crash**.

This is where the concept of **exception handling** plays the role of a smart and elegant safety net. Instead of letting the program fall apart when a problem occurs, we give backup instructions to the computer: "*Hey, if bad thing A happens, do rescue action B, and stay alive, don't die please! :(*" This isn't just about preventing app crashes, but about building user trust that our application is tough enough to weather any technical storm politely and under control.

1.2 Formal Definition

In Java and OOP terminology, **exception handling** is a structured mechanism for handling anomalies or abnormal conditions that arise during program execution (runtime). An exception itself is technically an object created by the **Java Virtual Machine (JVM)** when an error occurs, wrapping detailed information about what went wrong, where the error is located, and the sequence of events (**stack trace**).

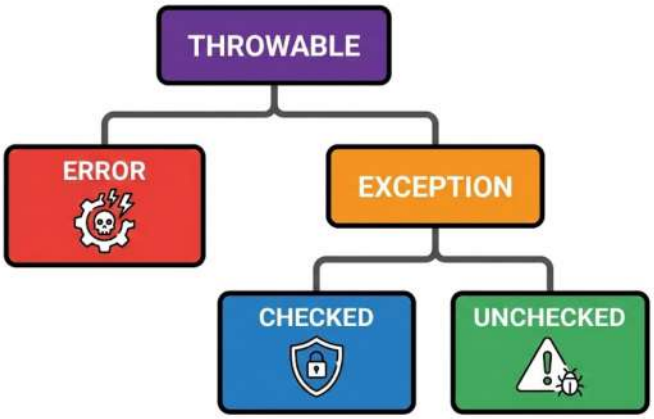
This handling system works by separating the main logic code from the error handling code, keeping the program flow clean and readable. Formally, this mechanism involves the process of "**throwing**" an exception object when a problem is detected, and "**catching**" that object elsewhere to be processed according to a predetermined recovery procedure. By applying this, we ensure data integrity remains intact and the program can continue execution or at least stop gracefully (**graceful shutdown**).

1.3 Concepts Elaboration

1.3.1 Exception Hierarchy

Before handling errors, we must first recognize the hierarchy of problems in Java. At the very top sits the `Throwable` class, a class that acts as an alarm for all problems that

have ever occurred in a program—it is the ancestor of everything that can be "thrown" out of the normal program flow. From here, the family splits into two main branches with opposing philosophies: `Error` and `Exception`.



In this module, we will not discuss `Throwable` and `Error` in depth. Instead, we will focus on `Exception` as the foundation. Beyond that, you can explore advanced topics related to exceptions on your own.

Error

Up until now, you probably thought that all problems in a program are errors, right? Not entirely wrong, but not quite accurate either. In truth, the term "Error" should be used more appropriately once you understand its concept.

Error is a failure/problem that occurs at a higher level (not within the program you are currently writing), ranging from software-level issues (IDE, operating system, etc.) to hardware-level issues (memory management failure, CPU damage, and so on). The easiest way to identify an Error is by its name, which always ends with "...Error", like running out of computer memory (`OutOfMemoryError`), a broken Java engine (`InternalError`), etc.

The key here is you are strictly forbidden from trying to handle errors because this usually means there is a **physical problem** beyond your code's control. If the machine is already destroyed, there is no point in trying to fix it via code—the application must stop to prevent further damage. Once again, we will not delve deeper into this topic; we will only explore exceptions.

Example:

```java
//A method that calls it self
public static void randomMethod(){
    randomMethod();
}

//Main Method
public static void main(String[] args) {
    randomMethod(); //we run that method
}
```

Output:

```
ERROR!
Exception in thread "main" java.lang
    .StackOverflowError
    at Main.randomMethod(Main.java:13)
    at Main.randomMethod(Main.java:13)
    at Main.randomMethod(Main.java:13)
```

The event above will trigger a `StackOverflowError`, which is a type of failure at a higher level (system or memory).

Its hallmark is visible in the name, which always ends with "...Error." This belongs to the category of problems that are strictly **unrecoverable**, because the memory itself is corrupted or exhausted. The only solution is for the program to stop completely to prevent further damage to your computer system.

Exception

The exception—this is our focus. Unlike Error, an exception is any problem which are "human" problems arising from program logic or external interactions that we can "control." We can handle all exceptions from within the program code. Exceptions themselves also come in many types that you can explore on your own.

Example:

```java
int[] numberList = {10, 20, 30};
System.out.println("Fifth number: " + numberList[5]); //5th element? is it exist?
System.out.println("Program is still running safely :)");
```

Output:

```
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: Index 5 out of bounds for
    length 3
    at Main.main(Main.java:7)
```

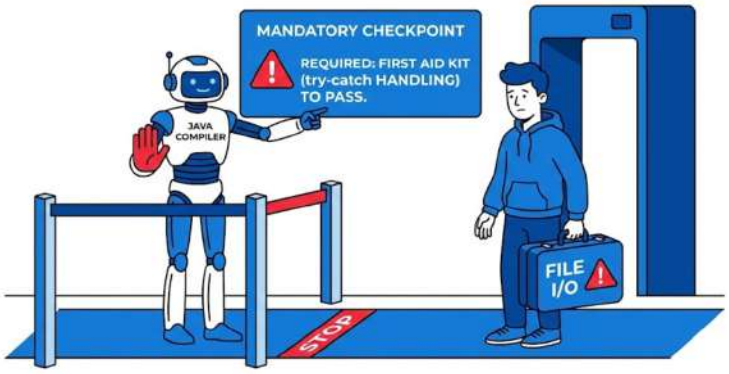
It's called an **exception** because the problem arises from application logic (incorrect data access) that is still recoverable, not a fatal system or memory failure.

- The program creates an array containing three numbers: `10, 20, 30`.
- Then there is a command to display the element at index 5.
- Since the array only has indices 0–2, accessing index 5 is invalid.
- Java immediately stops execution and displays an `ArrayIndexOutOfBoundsException` message
- The line of code after that command is never executed.
- Since this is an Exception, we can provide “recovery instructions” through a **try-catch** block so the program doesn’t crash abruptly. We will discuss the try-catch mechanism later.

1.3.2 Exceptions

Alright, now we have begun to dive into the concept of **exceptions**. This is the area that confuses beginners the most: "*Why do some errors make the program fail to compile, while others only appear silently when the program runs?*" Based on that question, the answer lies in two types of exceptions, namely **checked exceptions** and **unchecked exceptions.**

Checked Exception



Checked exceptions are risks that Java can predict from the start because they involve external factors, such as "file not found," "network disconnected," etc. Applying **try-catch** is mandatory for this type of exception. The reason why it is required is because this type of exception represents a problem that is certain to potentially occur, and it is **not**

the programmer's fault. Java has already anticipated the problems that will arise and requires you to prepare a container to handle them (**try-catch**).

Example:

This example uses `FileReader`, where Java requires us to be prepared in case the file we're looking for doesn't exist.

```
try{
    FileReader file = new FileReader("secret_note.txt");
} catch (FileNotFoundException e){
    System.out.println("Oops, looks like the file is missing!");
}
```

Output:

```
Oops, looks like the file is missing!
```

In this case, we're trying to open a file named "secret_note.txt". Since this is a **checked exception**, Java is already "**suspicious**" from the start that the file might be missing or the name might be misspelled.

- The `try` block is used as a testing ground for commands that carry a certain level of risk
- The `catch` block acts as the rescue team. If the file is indeed not found, the program won't just crash; instead, it will execute the command inside `catch` to notify the user politely
- **NOTE: WE WILL LEARN try-catch LATER**

The next example below shows what happens if we're stubborn. We know the `FileReader` command is risky, but we haven't provided a try-catch or a `throws` declaration in the method.

```
FileReader file = new FileReader( fileName: "data.txt");
                  Unhandled exception: java.io.FileNotFoundException
```

Jreng... jreng... Jreng... As a result, this program will never even get to run. The IDE or compiler will show a **red error message** forcing you to handle the risk beforehand. This is why it's called "**checked,**" because it's checked by the compiler before the program even has a chance to run.

Unchecked Exception

On the other hand, unchecked exceptions (which are descendants of `RuntimeException`) usually happen due to our own mistakes as programmers. Classic examples are trying to divide a number by zero or accessing an array index that doesn't exist. Applying try-catch for unchecked exceptions is **optional**. This is because Java cannot predict a programmer's negligence in writing code. If we are uncertain about the code we have written, it is HIGHLY RECOMMENDED to place that code inside a try-catch.

Example:

This example demonstrates logic errors that usually happen due to oversight when writing code, such as dividing a number by zero.

```
int number = 10;
int divisor = 0;

int result = number / divisor;

System.out.println("Result: "+ result);
```

Output:

```
Exception in thread "main" java.lang.ArithmeticException: / by zero
    at Main.main(Main.java:12)
```

```
.ArithmeticException: / by zero
```

- Unlike the first example, Java won't protest when you write this code. You can compile it smoothly without any red warnings
- The trouble only starts when the program is running (runtime). Once it hits the division line, the program will suddenly stop and throw an `ArithmeticException`
- Why doesn't Java require a **try-catch** here? Because theoretically, this error could be avoided with proper logic (like checking if the divisor is zero first), so Java leaves the responsibility entirely to our diligence as programmers

Basic Comparison between Checked and Unchecked Exceptions:

- **Checked:** Mandatory handling (**try-catch** or **throws**), checked at compile-time, usually I/O or Database issues.
- **Unchecked:** Not mandatory to declare, checked at runtime, usually logic issues like `NullPointerException`.

1.3.3 Try-Catch-Finally

After repeatedly mentioning the term try-catch in the previous discussions, we are now finally beginning to explain its concept. Simply put, try-catch is a pair of code blocks used to guard against and capture exceptions. There is also the term `finally`, which can be paired with try-catch (**try-catch-finally**). Below is a detailed explanation of each block.

```
try {
    //Code that has the potential to throw exceptions
} catch (Exception e) {
    //This code will run if the code above actually throws an exception
} finally {
    //This code will always run, whether there are exceptions or not
}
```

- The `try` block is the **experiment zone** where you place code that risks "exploding."
- The `catch` block is the **emergency unit** ready to catch the Exception object if an explosion actually happens. You can have multiple `catch` blocks to handle different types of "explosions," ensuring each problem gets specific and targeted treatment.
- There is one more part often overlooked but vital—the `finally` block. This block is a faithful promise that Java will always execute, whether the program succeeds, an exception occurs, or even if you call a `return` command halfway through

Example without try-catch:

```
int data = 100 / 0; //uupsss!
System.out.println("This output always executed.");
```

Output:

```
Exception in thread "main" java.lang.ArithmeticException: / by zero
    at Main.main(Main.java:7)
```

Example with try-catch:

```
try {
    int data = 100 / 0; //uupsss!
    System.out.println("No Exceptions :)");
} catch (Exception e) {
    System.out.println("Exceptions occurs!!!");
} finally {
    System.out.println("This output always executed.");
}
```

Output:

```
Exceptions occurs!!!
This output always executed.
```

Got it? You already understand the benefits of using **try-catch**, right? If not, **I'll smack your head.**

- **Why Use Finally?:**Imagine you are borrowing a book from the library (opening a connection). Whether you finish reading the book or the book gets torn (an error), you must still return the book to the librarian (closing the connection). The `finally` block ensures you don't forget to "return" those resources to the system
- **Special Power:** Even if you write a `return` statement inside the `try` or `catch` block, Java will still take the time to run the `finally` contents before actually exiting the method.

Finally

We can also choose not to use the `finally` keyword, because `finally` is optional.

```
try {
    int data = 100 / 0; //uupsss!
    System.out.println("No Exceptions :)");

} catch (Exception e) {
    System.out.println("Exceptions occurs!!!");

}
```

Output:

```
Exceptions occurs!!!
```

Why not just use if-else?

You might ask, "*Why not just use if-else to check for errors?*" Use **if-else** for predictable conditions where you can control the flow, like validating if an input number is greater than zero. However, for exceptional events involving external systems or unexpected errors,

try-catch-finally is a cleaner solution so your main program logic doesn't get buried under a pile of confusing manual checks.

```
String inputUser = "123A"; // Invalid input because it contains a letter

try {
    int number = Integer.parseInt(inputUser);
    System.out.println("Inputted number: " + number);
} catch (NumberFormatException e) {
    System.out.println("Error: Input must be digital numbers only!");
}
```

In the example above, using **if-else** to ensure text is a valid number would require complex regular expression logic or manual character-by-character checking. **Try-catch** provides a much more **elegant shortcut** by letting Java perform its own internal validation, while we simply focus on preparing a backup plan if the format is wrong. This keeps the code concise and prevents the main application logic from "drowning" in a pile of exhausting technical checks.

Now, let's try use **if-else**:

```
String inputUser = "123A";
boolean isNumber = true;

//DAMNNNNN
for (int i = 0; i < inputUser.length(); i++) {
    if (!Character.isDigit(inputUser.charAt(i))) {
        isNumber = false;
        break;
    }
}

if (isNumber && !inputUser.isEmpty()) {
    int number = Integer.parseInt(inputUser);
    System.out.println("Inputted number: " + number);
} else {
    System.out.println("Error: Input must be digital numbers only!");
}
//I'm tired, I wanna sleep...
```

As you can see, even for simple validation, we have to create a loop and check characters one by one manually before daring to convert it to a number. Imagine if the validation were more complex; your code would be full of "garbage" checking logic that

obscures the core of your main program. That is why try-catch is highly recommended for handling situations where errors might occur but are difficult or inefficient to check manually one by one.

1.3.5 Custom Exception

Previously, we have discussed a lot about how to use exceptions, but now we will create our own exception. Basically, the exceptions we studied earlier are built-in to Java—we simply call them to use them.

However, the real world is complex. Java does not have built-in exceptions for specific business cases in our program. Built-in exceptions are only used for general problems in writing program code. For unique issues in the business logic of our programs, **custom exceptions** are needed to handle them.

How to Create Custom Exceptions

To create your own exception, you simply need to create a new class that extends the `Exception` class for types that must be handled (**checked**) or `RuntimeException` for more relaxed types (**unchecked**). By performing this inheritance, your custom class automatically inherits all the capabilities of a Java exception object, including carrying a custom message and recording the error location trail (**stack trace**).

Example:

```
class InvalidAgeException extends Exception {
    public InvalidAgeException(String message) {
        super(message);
    }
}
```

Once the exception class is defined, the next step is determining where that "**alarm**" will be sounded.

How to Use Custom Exceptions: throw (without 's')

The `throw` keyword is used explicitly to cast an exception object when a bad condition is detected inside the code. Imagine this as **pulling an emergency alarm lever**, as soon as this command is executed, the normal program flow stops, and the system starts looking for who can catch that alarm.

It is important to understand that `throw` is the primary instruction to trigger *any* exception in Java. You can use it to call your own custom exceptions or Java's built-in exceptions (like `IllegalArgumentException`) whenever your business logic detects an anomaly.

Example:

Here, we try to use the Custom Exception `InvalidAgeException` directly, without going through a method as before. You can see that the `throw` keyword is used.

```
int age = 15;

try {
    if (age < 18) {
        throw new InvalidAgeException("Sorry, you're still a kid");
    }
    System.out.println("next process...");
} catch (InvalidAgeException e) {
    System.out.println(e.getMessage());
}
```

Output:

```
Sorry, you're still a kid
```

How to Use Custom Exceptions: throws (with 's)

The `throws` keyword is placed in the **method signature** to announce to the outside world that the method **carries a risk**. This is an honest way of saying, *"This method might explode, so whoever calls it must be prepared with a try-catch block."* If you throw a checked exception inside a method but do not catch it there, you **must** list it using `throws`.

`throws` applies universally to all types of checked exceptions, whether they originate from the Java standard library (like **IOException**) or custom exceptions you define. This is a **"pass the responsibility"** mechanism ensuring that every risk always has a clear handling plan.

Example:

Here we try to create a method that implements the Custom Exception we previously defined (`InvalidAgeException`). The implementation of this Custom Exception is done using the `throws` keyword.

```
static void checkAgeRequirement(int age) throws InvalidAgeException {
    if (age < 18) {
        throw new InvalidAgeException("Sorry, you're still a kid");
    }
    System.out.println("Monggo...");
}
```

Next, we can use that method. Here, we try entering a number below 18 to trigger the exception.

```
public static void main(String[] args) {
    try {
        checkAgeRequirement(15);
    } catch (InvalidAgeException e) {
        System.out.println(e.getMessage());
    }
}
```

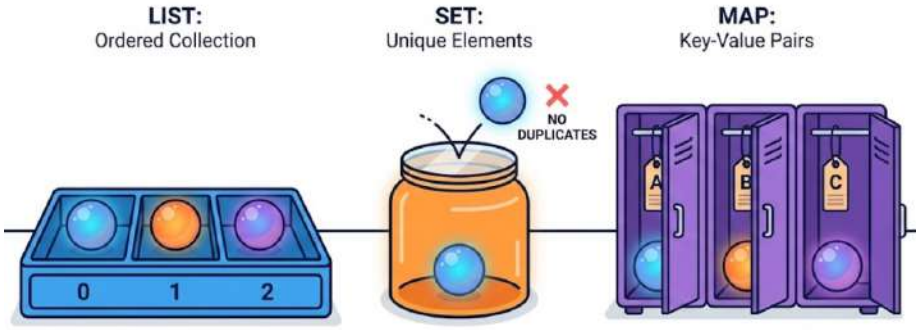
Output:

```
Sorry, you're still a kid
```

Full code:

```java
//Custom Exception
class InvalidAgeException extends Exception {
    public InvalidAgeException(String message) {
        super(message);
    }
}

class Main {
    //Method that implements the Custom Exception above
    static void checkAgeRequirement(int age) throws InvalidAgeException { //using throws
        if (age < 18) {
            throw new InvalidAgeException("Sorry, you're still a kid");
        }
        System.out.println("Monggo...");
    }

    //Main
    public static void main(String[] args) {
        try {
            //Use the method
            checkAgeRequirement(15);
        } catch (InvalidAgeException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

2. Java Collections



2.1 Why Should I Understand This?

Imagine you are building a digital shopping list application. Initially, you use a standard array to store items, but suddenly you realize a huge problem—arrays have a rigid, fixed size from the moment they are created. If you order a shelf for 5 items, but your friend suddenly asks to add a 6th item, you are forced to buy a new, larger shelf, move the old 5 items one

by one, and only then place the 6th item. A very exhausting and time-consuming process, right?

Even more headache-inducing: what if you want to ensure there are no duplicate items in the cart, or you want to find a specific item very quickly without checking the entire shelf from end to end? Doing all that manually using arrays and standard **for** loops is like trying to manage a giant warehouse alone without a computerized system. You would spend more time managing “how to store data” than focusing on your "business logic."

This is where the **Java Collections Framework** arrives as your super-genius warehouse assistant. It provides various types of ready-to-use "containers" that can shrink or grow automatically, guarantee no duplicate data, and even organize data so it is easy to search. By using Collections, you no longer need to worry about complex data structures because Java has provided solutions in a form that is neat, efficient, and ready to use.

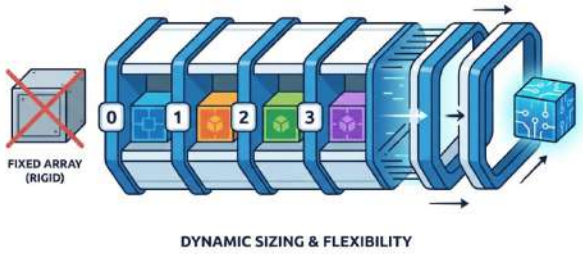
2.2 Formal Definition

Formally, the **Java Collections Framework (JCF)** is a unified architecture provided by Java to store and manipulate a group of objects. JCF is not just a collection of classes, but an ecosystem consisting of **interfaces**, **implementations**, and **algorithms** working harmoniously under the `java.util` package.

The core of JCF is providing standard data structures like `List`, `Set`, and `Map` that have been optimized for performance. Through the use of uniform interfaces, JCF allows developers to write flexible code where we can switch storage strategies (e.g., from `ArrayList` to `LinkedList`) without changing much of the main program logic. This supports OOP principles of abstraction and encapsulation, where technical storage details are hidden behind clean interface contracts.

2.3 Concepts Elaboration

2.3.1 ArrayList



You've heard the term "**array**" before, right? From the moment we first learn about arrays, we usually think they are the best solution for storing large amounts of data of the same data type. This method is more efficient than creating many separate variables for all that data.

However, Arrays also have their problems:

1. **Fixed size:** once defined, it cannot be changed. If we suddenly receive more data than the Array's size, we are forced to create a new Array variable.
2. **Memory waste:** sometimes we anticipate by preparing an Array with 100 slots, but the incoming data only fills 10 slots, leaving 90 slots unused.
3. **Homogeneous data type:** Arrays cannot store multiple data types at once.

From these problems, `ArrayList` was born as the solution. `ArrayList` can do what a regular Array variable cannot.

Actually, `ArrayList` is not the only solution option—there are alternatives such as `LinkedList`, `HashSet`, `HashMap`, and others. All of these terms have also been mentioned in the Formal Definition section earlier. For this module, we will focus only on discussing `ArrayList` as the foundation of your understanding. **FEEL FREE TO EXPLORE THE REMAINING TERMS BY YOURSELF!**

Imagine a row of chairs that can magically add its own seats when a new guest arrives, and remove empty seats to keep the room tidy. This is the main advantage of an `ArrayList`—it is a smart version of a conventional array where storage capacity can stretch and shrink automatically as needed.

ArrayList Declaration

Before declaring an ArrayList, we need to import the library.

```
import java.util.ArrayList;
```

After that, we can declare it right away. Although there are several ways to declare an `ArrayList`, in this lesson we'll focus on one of the simplest and safest approaches.

```
ArrayList<DataType> varName = new ArrayList<>();
```

The code above is a declaration of a variable of type `ArrayList` named `varName`.

- `ArrayList<DataType>`:
  - `ArrayList`: is a data structure in Java used to store a collection of elements dynamically (its size can grow or shrink).
  - `<DataType>`: commonly known as *Generics*. Whatever you put inside the angle brackets `<>` will become the data type for all values in this ArrayList variable. Oh, and you cannot use primitive data types (like `int`, `char`, `boolean`, etc.) in generics. You can only use objects that represent those

data types, which are called **Wrapper Classes (like** `Integer`**,** `Character`**,** `Boolean`**, etc.)**.

- `varName`**:** This is just the variable's name. You can change it to anything you like, because it's simply the label for this ArrayList. Later, whenever you want to use any ArrayList method to do a certain operation, you'll call this name. That's why it's important to give it a clear and suitable name.
- `new ArrayList<>()`**:**
  - Creates (initializes) a new `ArrayList` object in memory.
  - The empty `<>` means it uses the generic type already defined.

Alright, after understanding how to declare an `ArrayList`, now it's time to perform various operations that can be done using the available ArrayList **methods**, let's go straight into the explanation of each method in `ArrayList`.

Adding and Modifying Data

Methods in this category are used to insert new elements or update existing ones.

Method	Description	Example
`add(E element)`	Add an element to the end of the list.	`list.add("Anu");`
`add(int index, E element)`	Insert an element at a specific position (index).	`list.add(1, "Uni");`
`set(int index, E element)`	Replace the element at a specific index with a new element.	`list.set(0, "Ono");`
`addAll(Collection c)`	Add all elements from another collection into the list.	`list.addAll(anotherList);`

Example: `add(int index)`

```
ArrayList<String> animals = new ArrayList<>();

animals.add("Cat");
animals.add("Dog");

System.out.println(animals);
```

Output:

```
[Cat, Dog]
```

Example: `add(int index, E element)`

```
ArrayList<String> animals = new ArrayList<>();

animals.add("Cat");
animals.add("Dog");

// Inserting "Spiderman" at index 1 (second position)
animals.add(1, "Spiderman");

System.out.println(animals);
```

Output:

```
[Cat, Spiderman, Dog]
```

Example: `set(int index, E element)`

```
ArrayList<String> animals = new ArrayList<>();

animals.add("Cat");
animals.add("Dog");

// Replacing "Cat" (index 0) with "Bird"
animals.set(0, "Bird");

System.out.println(animals);
```

Output:

```
[Bird, Dog]
```

Example: `addAll(Collection c)`

```
ArrayList<String> teamA = new ArrayList<>();
teamA.add("Alice");

ArrayList<String> teamB = new ArrayList<>();
teamB.add("Bob");
teamB.add("Charlie");

// Adding all members of teamB to teamA
teamA.addAll(teamB);

System.out.println(teamA);
```

Output:

```
[Alice, Bob, Charlie]
```

Alright guys, maybe only for this table we'll provide an implementation example. For the next tables, please explore them on your own. We won't give more examples (**You have to be independent, dude**).

Retrieving and Searching Data

Use these methods to access the contents of a list or to find the position of an object.

Method	Description	Return
`get(int index)`	Retrieve the element at a specific index.	Object at that index.

Method	Description	Return
`indexOf(Object o)`	Find the first index of the specified object.	`int (index)` or `-1` if not found.
`lastIndexOf(Object o)`	Find the last index of the same object.	`int` (index)
`contains(Object o)`	Check whether an object exists in the list.	`true/false`

Deleting Data

Methods for clearing or reducing elements in a list.

Method	Description	Example
`remove(int index)`	Remove the element at a specific index position.	`list.remove(0);`
`remove(Object o)`	Remove the first element that matches the specified object.	`list.remove("Java");`
`clear()`	Remove all elements from the list (clear the list).	`list.clear();`

Information and Utilities

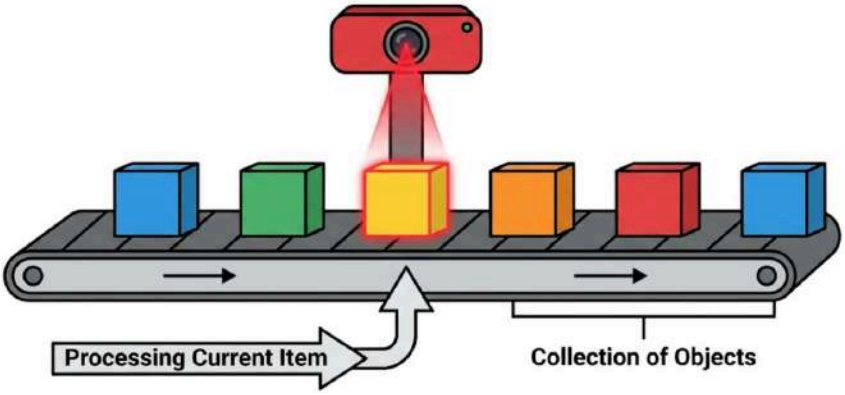
Additional methods to check the status of a list or to transform its structure.

Method	Description	Example
`size()`	Return the number of elements in the list.	`int total = list.size();`
`isEmpty()`	Check whether the list has no elements.	`if (list.isEmpty()) { ... }`
`toArray()`	Convert the ArrayList into a regular array.	`Object[] arr = list.toArray();`
`sort(Comparator c)`	Sort the elements (using **null** for natural ordering).	`list.sort(null);`

2.3.2 Iterator & Enhanced For-Loop

Now that you've learned how to declare an ArrayList and use its methods, let's move on to **traversing** the data inside it. In Java, there are generally two ways to go through an ArrayList: using a **for-each loop** or an **Iterator**.

Enhanced For-Loop (For-Each)



Often called **for-each**, this is the most popular and neatest way to explore a collection. Visualize this like a librarian walking from one end of the shelf to the other, taking each book one by one and reading it, without needing to know which coordinate the book is located at. The syntax is very clean and almost resembles human language.

Here is the syntax:

```
for (type elementName : arrayName) {
    // code block to be executed
}
```

Well… it's just like a for-loop, the only difference is inside the parentheses. Here's what goes inside those parentheses.

- `Type`**:** This is the data type of each element in the **array** that we will explore. Make sure you write the same data type as your **array's** data type.
- `elementName`**:** This is the alias or name we assign to each element we encounter while exploring the contents of the **array**. You're free to choose the name, but my suggestion is to keep it short and relevant, since you'll use this name inside the **for-each** block (if you want to).
- `arrayName`**:** This is the name of the **array** you want to iterate through.

Example:

First, let's try using **for-each** to explore the contents of a **regular array** variable.

```java
//Array Variable
int[] arrayX = {80, 90, 75, 100};

//Exploring all elements of the array
for (int each_element : arrayX) {
    System.out.println(each_element);
}
```

Output:

```
--- Regular Array ---
80
90
75
100
```

The breakdown:

- We set the type as `int` because every element in `arrayX` is of an integer data type.
- We name the element `each_element`, but again, you are free to choose any name you like.
- We call `arrayX` because, well, that's the array we want to explore.

Now, let's try to explore an `ArrayList`, not an `Array`.

```java
//ArrayList Variabel Declaration
ArrayList<Integer> arrayListX = new ArrayList<>();

//fill element to ArrayList
arrayListX.add(200);
arrayListX.add(300);
arrayListX.add(140);
arrayListX.add(500);
arrayListX.add(660);

//Exploring all elements of the ArrayList
for (int each_element : arrayListX) {
    System.out.println(each_element);
}
```

Output:

```
--- ArrayList ---
200
300
140
500
660
```

Using for-each with an `ArrayList` is actually pretty similar to using it with a regular array.

Well, actually for-each has a limitation, it cannot modify array data in real time while the loop is running. If we force a modification—for example, by using the `add()` method on an `ArrayList` during the loop—Java will throw a `ConcurrentModificationException`.

```java
//ArrayList Variabel Declaration
ArrayList<Integer> arrayListX = new ArrayList<>();
int counter = 1;

//fill element to ArrayList
arrayListX.add(200);
arrayListX.add(300);
arrayListX.add(140);
arrayListX.add(500);
arrayListX.add(660);

//Exploring all elements of the ArrayList
for (int each_element : arrayListX) {
    //here we adding new data to the arrayListX during loop
    if (counter == 4){
        arrayListX.add(400);
    }

    System.out.println(each_element);
    counter++;
}
```

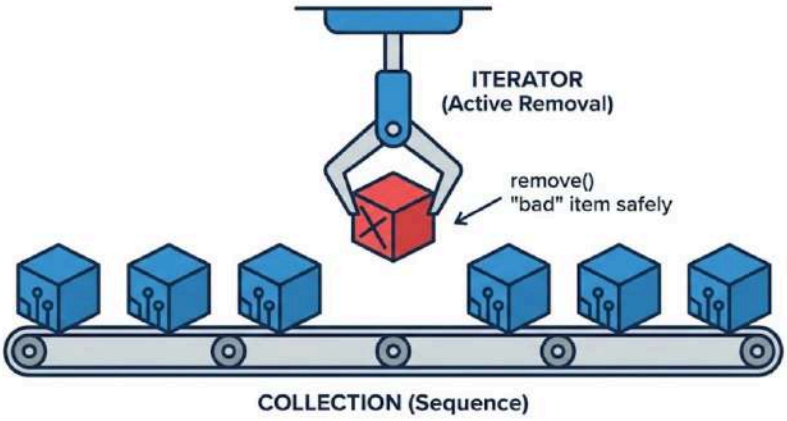
Output:

```
--- ArrayList ---
200
300
140
500
ERROR!
Exception in thread "main" java.util.ConcurrentModificationException
    at java.base/java.util.ArrayList$Itr.checkForComodification(ArrayList.java:1095)
    at java.base/java.util.ArrayList$Itr.next(ArrayList.java:1049)
    at Main.main(Main.java:45)
```

Sooo… use this method if you only want to **read or process data** without changing the data structure (like adding or removing elements). Because of its simplicity, your code will look much more professional and easier for others to understand.

Iterator

If **for-each** can only be used to explore each element in an `ArrayList`, then Iterator comes as both an **explorer** and a **modifier** of the data elements in the `ArrayList`.



Have you ever played **Minecraft**? The simplest analogy is that **for-each** is like ***adventure mode***, you can only walk around without breaking or placing blocks, while Iterator is like ***survival/creative mode***, where you can not only explore but also break and place blocks.

Here is the syntax for declaring an Iterator.

```
Iterator<DataType> iteratorName = arrayListName.iterator();
```

You're probably confused, because in the **for-each** before we didn't declare an object like this. You might also think that this declaration looks similar to declaring an `ArrayList`, right?

Although both serve as data explorers, **for-each** and Iterator differ in their form. Basically, **for-each** is a statement (**a loop statement**), while **Iterator** is an **object**. Since **Iterator** is an object, we are required to declare it so that we can perform various **Iterator** operations through that object.

Actually, Iterator has many methods, but there are three methods that beginners will often use when learning:

Method	Description	Return
`next()`	Returns the next element in the iteration.	The element type of the collection (**E**).
`hasNext()`	Checks whether the iteration has more elements.	true/false
`remove()`	Removes the last element returned by `next()` from the underlying collection.	Removes data in the current index.

For the other methods, feel free to explore them on the Internet or with popular AI models available today.

Example 1: `hastNext()`

First, let's try the `hasNext()` method. We will use this method to check whether the ArrayList has any contents or not.

```
//ArrayList Declaration
ArrayList<Integer> arrayListX = new ArrayList<>();
arrayListX.add(200);
arrayListX.add(300);
arrayListX.add(140);
arrayListX.add(500);
arrayListX.add(660);

//Iterator Declaration
Iterator<Integer> iteratorX = arrayListX.iterator();

// hasNext() will return true because there is still data ahead of the cursor
if (iteratorX.hasNext()) {
    System.out.println("There is still more data!");
} else {
    System.out.println("This ArrayList is empty");
}
```

Output:

```
There is still more data!
```

If we empty that `ArrayList` variable (no data at all), then `hasNext()` will return **false**, and the output will be *"This ArrayList is empty"*.

Example 2: `next()`

Here we try to retrieve the data from the `ArrayList` one by one to display it.

```java
//ArrayList Declaration
ArrayList<Integer> arrayListX = new ArrayList<>();
arrayListX.add(200);
arrayListX.add(300);
arrayListX.add(140);
arrayListX.add(500);
arrayListX.add(660);

//Iterator Declaration
Iterator<Integer> iteratorX = arrayListX.iterator();

//first use of next()
int element = iteratorX.next();
System.out.println("Data just retrieved: " + element);

//second use of next()
element = iteratorX.next();
System.out.println("Data just retrieved: " + element);

//third use of next()
element = iteratorX.next();
System.out.println("Data just retrieved: " + element);
```

Output:

```
Data just retrieved: 200
Data just retrieved: 300
Data just retrieved: 140
```

See? Here we run the same code repeatedly, but it produces different outputs each time. If you look closely, the values displayed follow the order of the data in the `ArrayList`. All of this is thanks to the `next()`.

Example 2: `remove()`

Well, this might be one of the fun parts. We've already filled the `ArrayList` with 5 integer data items, and by using the `remove()` method we'll try to delete all integers with values greater than 200.

```java
//ArrayList Declaration
ArrayList<Integer> arrayListX = new ArrayList<>();
arrayListX.add(200);
arrayListX.add(300);
arrayListX.add(140);
arrayListX.add(500);
arrayListX.add(660);

//Iterator Declaration
Iterator<Integer> iteratorX = arrayListX.iterator();

while (iteratorX.hasNext()) { //use hasNext()
    int value = iteratorX.next();

    if (value > 200) {
        // Removes values that greater than 200 from the original list
        iteratorX.remove();
        System.out.println("Values greater than 200 have been removed.");
    }
}

System.out.println("\nRemaining data:");

//Display all ArrayList's data after using the remove() method
for (int i : arrayListX){
    System.out.println(i);
}
```

Output:

```
Values greater than 200 have been removed.
Values greater than 200 have been removed.
Values greater than 200 have been removed.

Remaining data:
200
140
```

Let's break it down:

- In this code, we use `hasNext()` inside the parentheses of a while-loop as a condition. Since the `ArrayList` currently contains 5 integer data items, `iteratorX.hasNext()` returns `true`, so the code inside the while-loop is executed

- Next, we use the `next()` method to target the next data element for an operation—in this case, we apply the `remove()` method to that element
- Finally, we use an if-statement to filter which of the next elements have integer values greater than 200. When such a value is found, `iteratorX.remove()` takes action and immediately deletes that data element
- As a result, two data items remain in the `ArrayList`, namely 200 and 140, because neither of them is greater than 200

Fun, isn't it? Go ahead and try out the other methods from Iterator, **FEEL FREE TO EXPERIMENT AND PRACTICING!!!**

When to Choose Which?

Use Enhanced For-Loop (For-each) if:

- You only need to read (read-only) or display data.
- There is no plan to delete or add elements in the middle of the loop.
- You want code that is very clean and readable (**clean code**).

Use Iterator if:

- You need to delete elements (`remove`) while the iteration is ongoing.
- You are working with collections that do not have indices (like `Set`).
- You need more control (like moving backward and forward with `ListIterator`).

EXERCISE

Continue the previous work. Now the game can have more than 1 character using `ArrayList`, and we will use exceptions to handle errors. Here's what you need to do:

1. Declare an `ArrayList` variable so it can contain characters

```
List<Character> party = new ArrayList<>();
```

And here's how to place an class variable into it

```
party.add(newHero);
```

2. Make your own Exception (CustomException)

```
public class InvalidGameRuleException extends Exception {
    public InvalidGameRuleException(String message) {
        super(message);
    }
}
```

3. Use this on age restricted earlier and default switch case

```
try {
                // --- INPUT DATA DASAR ---
                System.out.print("Insert Name: ");
                String name = scanner.nextLine();

                // --- EXCEPTION HANDLING (Input Mismatch) ---
                System.out.print("Insert Age: ");
                // Jika user mengetik huruf, baris ini akan melempar
  InputMismatchException
                int age = scanner.nextInt();
                scanner.nextLine(); // Konsumsi newline

                // --- EXCEPTION HANDLING (Custom Exception) ---
                if (age < 13) {
                    throw new InvalidGameRuleException("Player is too young to play!
  Minimum age is 13.");
                }

                // --- PENGGUNAAN ENUM ---
                Gender gender = null;
                while (gender == null) {
                    System.out.print("Gender (M/F): ");
                    String gInput = scanner.nextLine();
                    if (gInput.equalsIgnoreCase("M")) gender = Gender.MALE;
                    else if (gInput.equalsIgnoreCase("F")) gender = Gender.FEMALE;
                    else System.out.println("Invalid input. Please type M or F.");
                }
```

```
Character newHero = null;

                switch (choice) {
                    case 1:
                        newHero = new Warrior(name, age, gender, "Sword", "Knight");
                        break;
                    case 2:
                        newHero = new Mage(name, age, gender, "Fire", 100);
                        break;
                    case 3:
                        newHero = new Archer(name, age, gender, 10, "Longbow");
                        break;
                    default:
                        // Melempar exception jika pilihan salah
                        throw new InvalidGameRuleException("Invalid class selection (Must be 1-3).");
                }

                // Menambahkan hero ke dalam LIST (Collection)
                party.add(newHero);
                System.out.println("Hero " + name + " added to the party!");

            } catch (InputMismatchException e) {
                // Menangkap error jika user memasukkan huruf saat diminta angka
                System.out.println("ERROR: Input must be a number! Please try again.");
                scanner.nextLine(); // Membersihkan buffer scanner agar tidak looping infinite
            } catch (InvalidGameRuleException e) {
                // Menangkap error custom (aturan game)
                System.out.println("GAME RULE ERROR: " + e.getMessage());
            } catch (Exception e) {
                // Menangkap error umum lainnya
                System.out.println("UNKNOWN ERROR: " + e.getMessage());
            }

            // Opsi menambah hero lagi
            System.out.print("\nAdd another hero? (y/n): ");
            String ans = scanner.nextLine();
            if (!ans.equalsIgnoreCase("y")) {
                addMore = false;
            }
        }
```

4. Add more exception and use it on Character Child classes

Here's what you should understand:

1. What's an `ArrayList`?
2. Why do we need an `ArrayList`? Isn't it the same as the usual array?
3. What's the benefit of using `ArrayList`?
4. What's an exception?
5. Name 3 exception in java

ASSIGNMENT

Smart City now can add data (Create) and see data (Read). But some administration in the city need a feature to delete data and unique validation (2 buildings can't have the same name). Here's what you need to do:

1. Add an custom exception

```
// Modul 5: Custom Exception
public class InvalidDataException extends Exception {
    public InvalidDataException(String message) {
        super(message);
    }
}
```

2. Implement this Exception in the Building class

```
// Tambahkan import exception jika beda package (disini asumsi satu
folder)
public abstract class Building {
    protected String name;
    protected String address;
    protected int numberOfFloors;
    protected BuildingStatus status;

    // UPDATE: Menambahkan 'throws InvalidDataException'
    public Building(String name, String address, int numberOfFloors,
BuildingStatus status) throws InvalidDataException {
        // Validate : name must not be empty
        if (name == null || name.trim().isEmpty()) {
            //Todo: Throw exception here
        }

        // validate : number must not be a minus
        if (numberOfFloors <= 0) {
            //Todo: Throw exception here
        }

        this.name = name;
        this.address = address;
        this.numberOfFloors = numberOfFloors;
        this.status = status;
    }

    public abstract void showBuildings();

    public String getName() { return name; }
}
```

3. Use foreach to read all the data on `ArrayList` (must have a default data before) and if the user inputs the same data on array, throw an exception.

`Main.Java`

```
private static void addHospital(Scanner scanner, ArrayList<Building>
list) {
   try {
       System.out.println("\n--- Add New Hospital ---");
       System.out.print("Name: ");
       String name = scanner.nextLine();

       System.out.print("Address: ");
       String address = scanner.nextLine();

       System.out.print("Floors: ");
       int floors = scanner.nextInt();

       System.out.print("Beds: ");
       int beds = scanner.nextInt();
       scanner.nextLine(); // Buffer

       BuildingStatus status = selectStatus(scanner);

       if(CheckDuplicate(list, name)){
           Hospital h = new Hospital(name, address, floors, status,
beds);
           list.add(h);
           System.out.println(">> Success: Hospital added.");
       } else {
           //Throw Exception
       }

   } catch (InputMismatchException e) {
       System.out.println("[ERROR] Format input salah (masukkan
angka untuk lantai/bed).");
       scanner.nextLine(); // Clear buffer
   } catch (InvalidDataException e) {
       // Modul 5: Custom Exception Handling
       System.out.println("[VALIDATION FAILED] " + e.getMessage());
   }
}
```

```
private static boolean CheckDuplicate(ArrayList<Building> list,
String nameBuilding){
   //Use ForEach to check if the name has dupliacte
```

4. Use an exception to handle if the input was not null and the input is correct (integer must be a number, not alphabet).

Example:

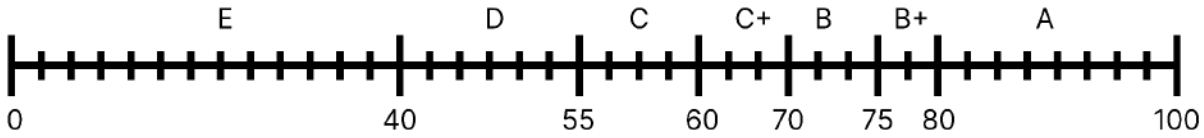
Main.Java

```
private static void addMarket(Scanner scanner, ArrayList<Building>
list) {
   try {
        System.out.println("\n--- Add New Market ---");
        System.out.print("Name: ");
        String name = scanner.nextLine();

        System.out.print("Address: ");
        String address = scanner.nextLine();

        System.out.print("Floors: ");
        int floors = scanner.nextInt();

        System.out.print("Revenue: ");
        double revenue = scanner.nextDouble();
        scanner.nextLine();

        BuildingStatus status = selectStatus(scanner);
        if(CheckDuplicate(list, name)) {
            Market m = new Market(name, address, floors, status,
revenue);
            list.add(m);
            System.out.println(">> Success: Market added.");
        } else {
            throw new InvalidDataException("[VALIDATION FAILED]
Duplicate market name found.");
        }
   } //Todo: Catch exception if the input was incorrect (integer, but
the user input it a string)
//Todo: implement Custom Exception
}
```

ASSESSMENT

ASSESSMENT ASPECT	POINT
Codelab	**20%**
Assignment	**80%**
Code Running Smoothly	10%
Understanding the Module	70%
Total	**100%**

Percentage Assessment



A = (81 - 100) → You're The Chosen One
B+ = (75 - 80) → Great!
B = (70 - 74) → Good!
67 = (67) → Square root of 4489
C+ = (60 - 69) → Keep it up!
C = (55 - 59) → Almost
D = (41 - 54) → Study Some More!
E = (0 - 40) →

Closing Statement?

Seeing red in the console output? Don't lose it, just handle that like a gentleman. Problems are meant to be handled, not ghosted and slept on.

Luckily, ArrayList is ready to contain everything for you, except maybe your life's baggage.

LET'S GO! ONE MORE MODULE AND WE ARE FREEEEEEEEEE!!!